

Learn to trade via direct reinforcement

John Moody, Matthew Saffell

IEEE Transactions on Neural Networks, Vol. 12, No. 4, July 2001, PP.
875-889

Presenter: Hung-Hsi Chen

Date: Sept. 1, 2009

Abstract

We present methods for optimizing portfolios, asset allocations and trading systems based on Direct Reinforcement. In this approach, investment decision making is viewed as a stochastic control problem, and strategies are discovered directly. We present an adaptive algorithm call **Recurrent Reinforcement Learning(RRL) for discovering investment policies**. The need to build forecasting models is eliminated, and better trading performance is obtained.

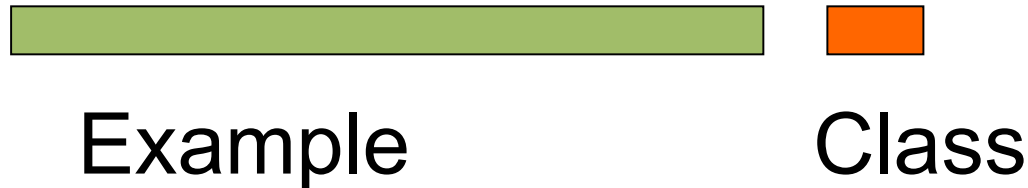
The Direct Reinforcement approach differs from dynamic programming and reinforcement algorithms such as TD-learning and Q-learning which attempt to estimate a value function for the control problem.

We find that the RRL Direct Reinforcement framework enables a simpler problem representation, avoids Bellman's curse of dimensionality and offers compelling advantages in efficiency.

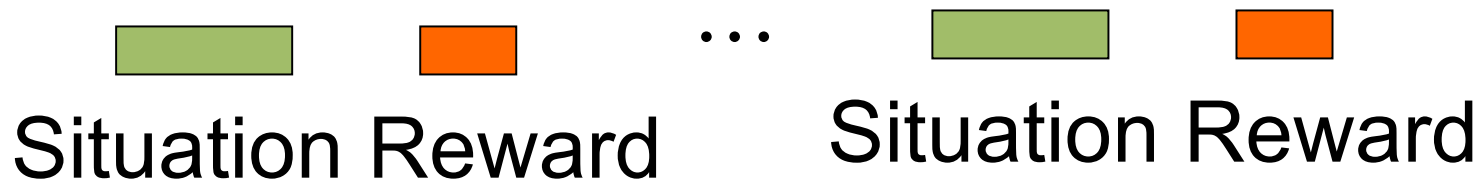
We demonstrate how Direct reinforcement can be used to optimize risk-adjusted investment returns (including the differential Sharpe ratio), while accounting for the effects of transaction costs. In extensive simulation work using real financial data, we find that our approach based on RRL produces better trading strategies than systems utilizing Q-learning (a value function method). Real world applications include an intra-daily currency trader and a monthly asset allocation system for the S&P 500 Stock Index and T-Bills.

Supervised & Reinforcement learning

Supervised Learning(learning from example):

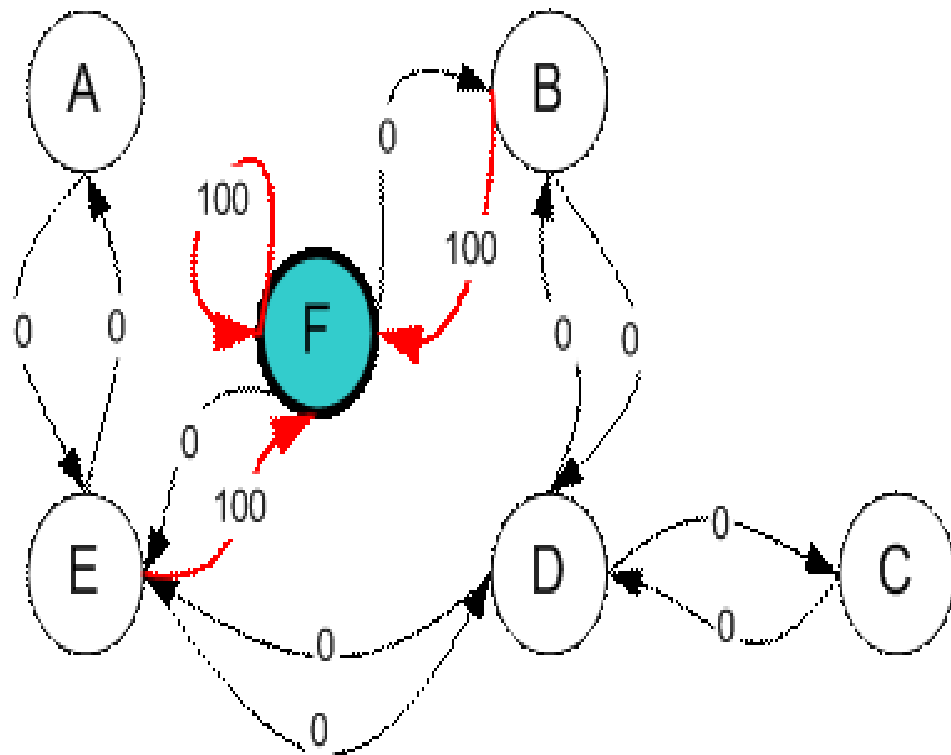


Reinforcement Learning(learning from reward):

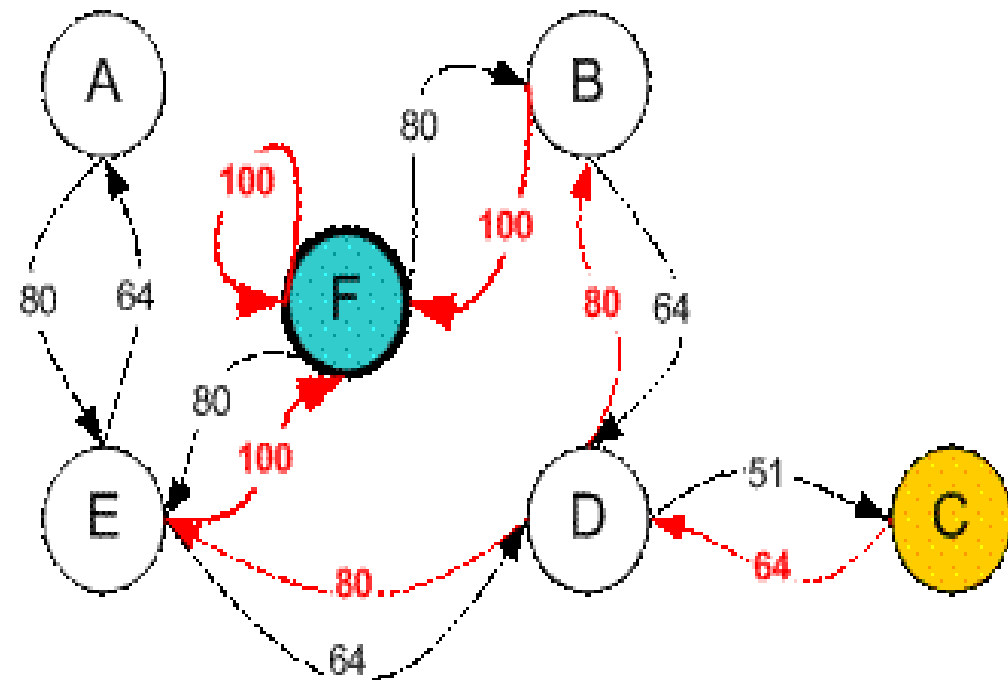


Reinforcement learning example (Q learning algorithm)

Input :



output :



Initialize $Q(s, a)$ arbitrarily

Repeat (for each episode)

Initialize s

Repeat (for each step of episode)

Choose a from s using policy derived from Q (e.g., ϵ -greedy)

Take action a , observe r, s'

$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$

$s \leftarrow s'$

Until s is terminal

$$\gamma = 0.8 \quad \mathbf{Q} = \begin{array}{c|cccccc} & A & B & C & D & E & F \\ \hline A & 0 & 0 & 0 & 0 & 0 & 0 \\ B & 0 & 0 & 0 & 0 & 0 & 0 \\ C & 0 & 0 & 0 & 0 & 0 & 0 \\ D & 0 & 0 & 0 & 0 & 0 & 0 \\ E & 0 & 0 & 0 & 0 & 0 & 0 \\ F & 0 & 0 & 0 & 0 & 0 & 0 \end{array} \quad \mathbf{R} = \begin{array}{c|cccccc} state \backslash action & A & B & C & D & E & F \\ \hline A & - & - & - & - & 0 & - \\ B & - & - & - & 0 & - & 100 \\ C & - & - & - & 0 & - & - \\ D & - & 0 & 0 & - & 0 & - \\ E & 0 & - & - & 0 & - & 100 \\ F & - & 0 & - & - & 0 & 100 \end{array}$$

$$Q(state, action) = R(state, action) + \gamma \cdot \text{Max}[Q(next\ state, all\ actions)]$$

$$Q(B, F) = R(B, F) + 0.8 \cdot \text{Max}\{Q(F, B), Q(F, E), Q(F, F)\} = 100 + 0.8 \cdot 0 = 100$$

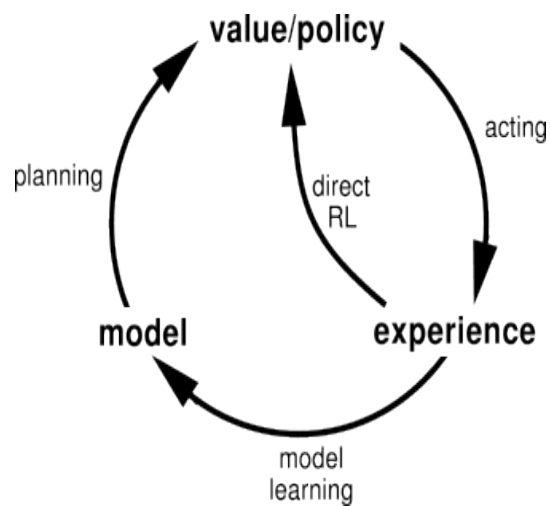
$$Q(state, action) = R(state, action) + \gamma \cdot \text{Max}[Q(next\ state, all\ actions)]$$

$$Q(D, B) = R(D, B) + 0.8 \cdot \text{Max}\{Q(B, D), Q(B, F)\} = 0 + 0.8 \cdot \text{Max}\{0, 100\} = 80$$

Finally we can get the result:

$$Q = \begin{array}{c|cccccc} state \backslash action & A & B & C & D & E & F \\ \hline A & - & - & - & - & 400 & - \\ B & - & - & - & 320 & - & 500 \\ C & - & - & - & 320 & - & - \\ D & - & 400 & 256 & - & 400 & - \\ E & 320 & - & - & 320 & - & 500 \\ F & - & 400 & - & - & 400 & 500 \end{array}$$

Reinforcement learning method



Value Learning

- Learns $V(state, action)$
- Optimal action is implicit.
- Ex: Q-Learning
- Suitable for delayed rewards

Direct Reinforcement Learning

- No value function
- Learns action policy directly
- Ex: Recurrent Reinforcement Learning (RRL) given by Moody
- Suitable in environments where rewards are immediate: Control problems, stock markets

Structure of trading systems

Our agent trades fixed quantities of a security z .

The price series is $\{z_1, z_2, \dots, z_t, \dots, z_T\}$ and corresponding price changes $r_t = z_t - z_{t-1}$

At each time step, our position is

$$F_t \in \{long, neutral, short\} = \{+1, 0, -1\}$$

We wish to learn a trading strategy $F_t = F(\theta_t; F_{i < t}, I_t)$

θ is the set of parameters we are learning

$I_t = \{z_{i \leq t}, \text{etc...}\}$ is our **Information** at time t

For example, a system with $m + 1$ "autoregressive inputs":

$$F_t = \text{sign}(uF_{t-1} + v_0r_t + v_1r_{t-1} + \cdots v_mr_{t-m} + w)$$

$$\theta = \{u, v_i, w\}$$

Whatever functional form used, $F(\theta)$ must be differentiable ($dF/d\theta$).

Profit and wealth

Daily Return: $R_t = r_t \cdot F_{t-1} - \delta |F_t - F_{t-1}|$
 δ is the transaction cost

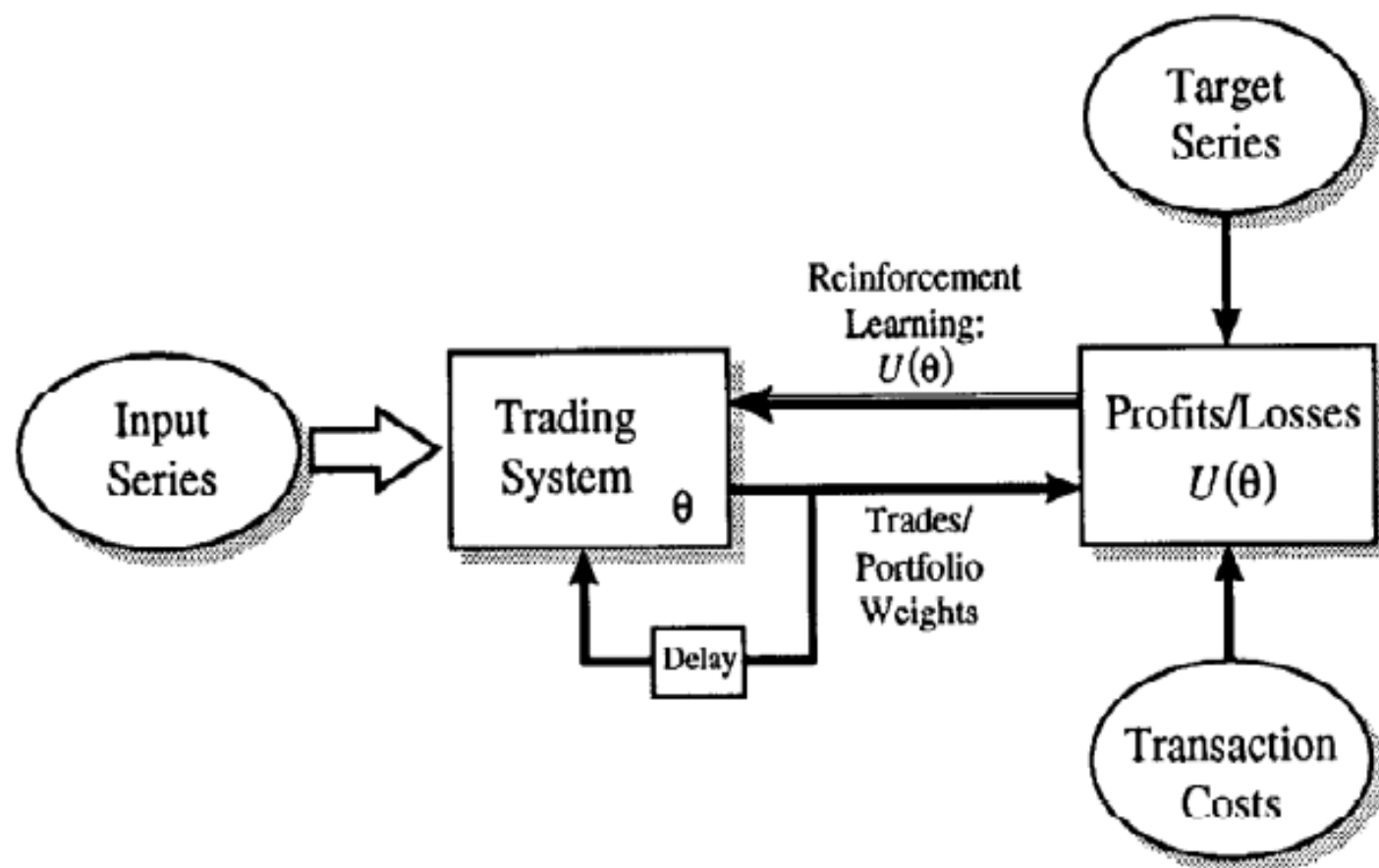
Total Profits: $P_T = \sum_{t=1}^T R_t$
i.e. no reinvesting

Wealth: $W_T = W_0 + P_T$

Utility: $U_T = U(R_1, \dots, R_T; W_0)$

Ex: $U_T = W_T$ measures profit without regard to risk

RRL “Direct Reinforcement” Approach



Experiment 1 : Artificial time series

Goal : show the system can learn

They generate log-normal random walks, but with autoregressive trends:

Trend variable: $\beta(t) = \alpha\beta(t-1) + \nu(t)$

Log price: $p(t) = p(t-1) + \beta(t-1) + \epsilon(t)$

$\epsilon(t)$, $\nu(t)$ are “noise” terms with zero mean, unit variance

$\alpha < 1$ sets how the “autoregressiveness”

$z_t = \exp(p(t))$ is used to generate 10,000-point price series.

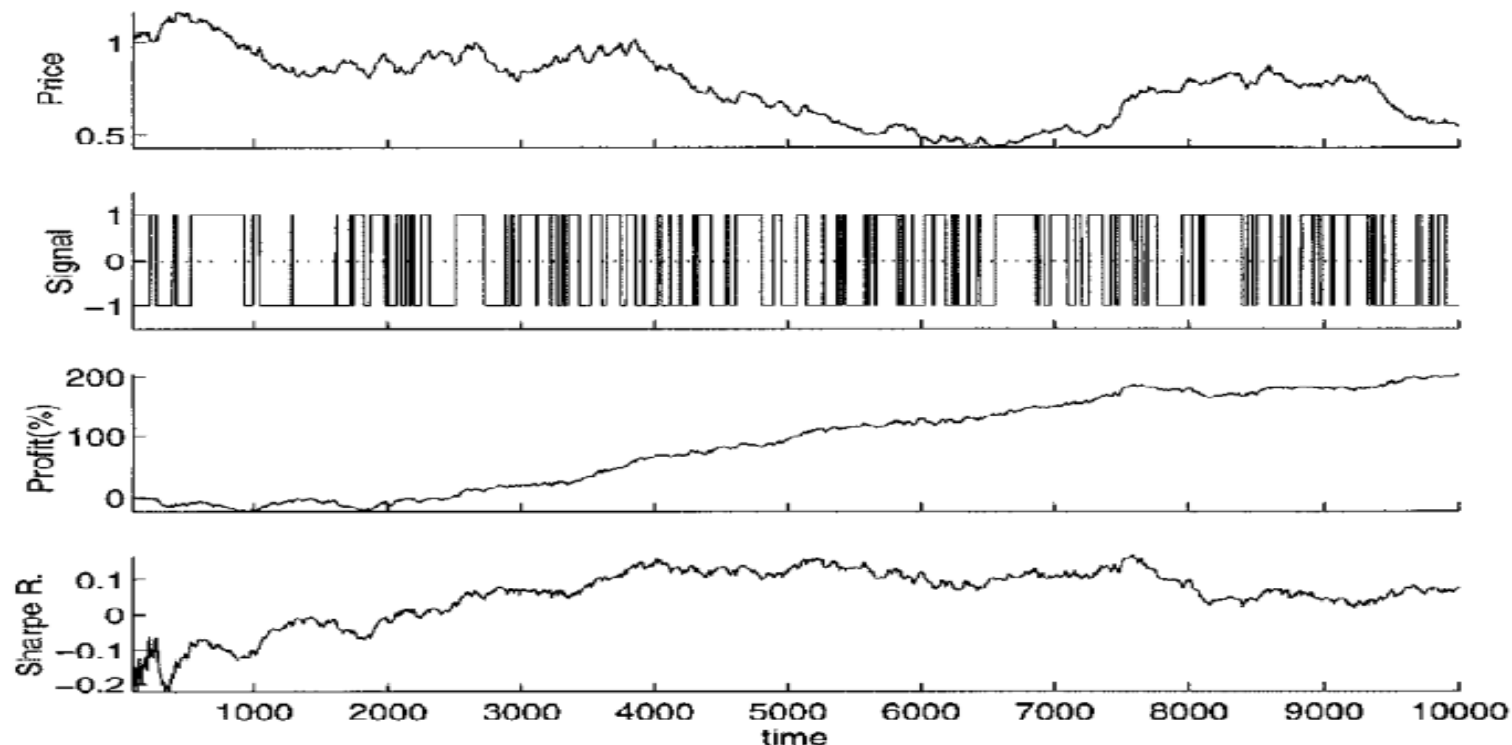
The trading function has autoregressive inputs (matches the data):

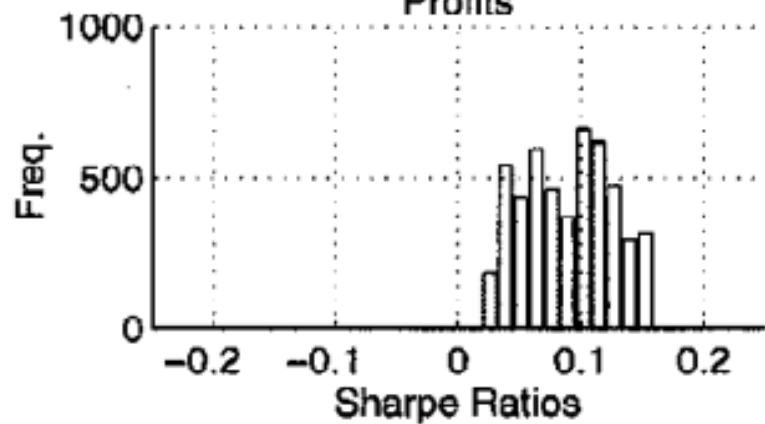
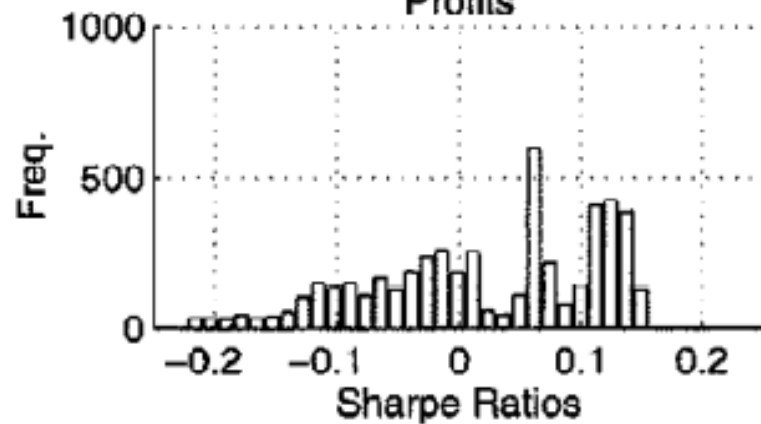
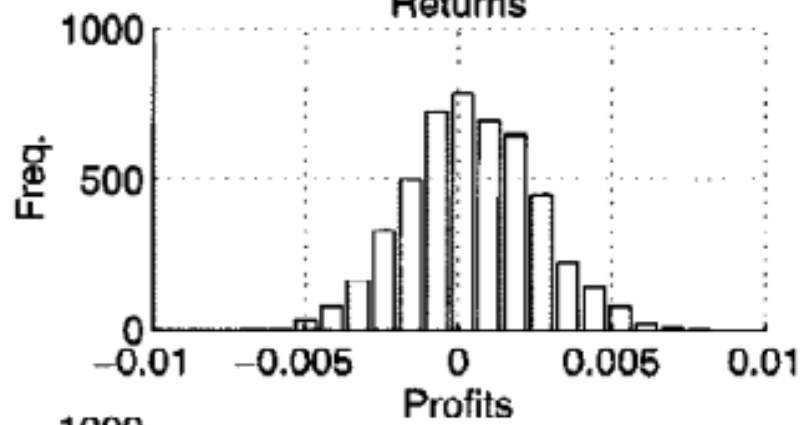
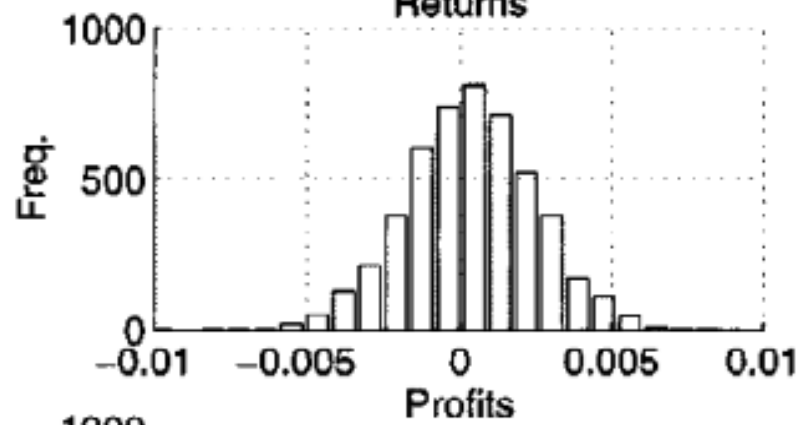
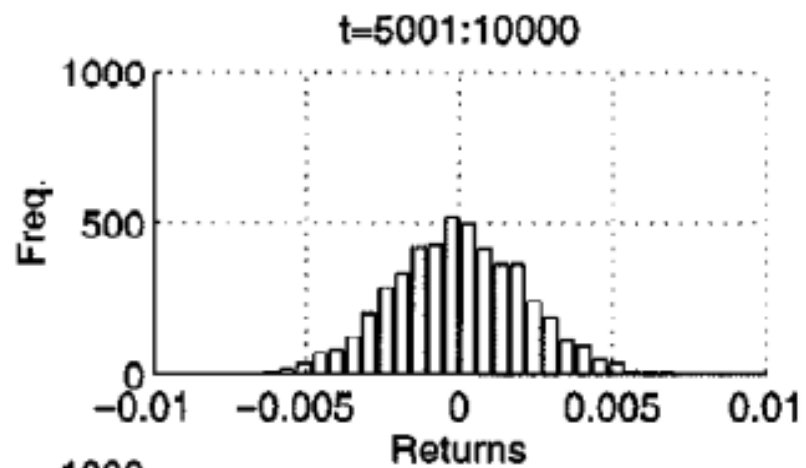
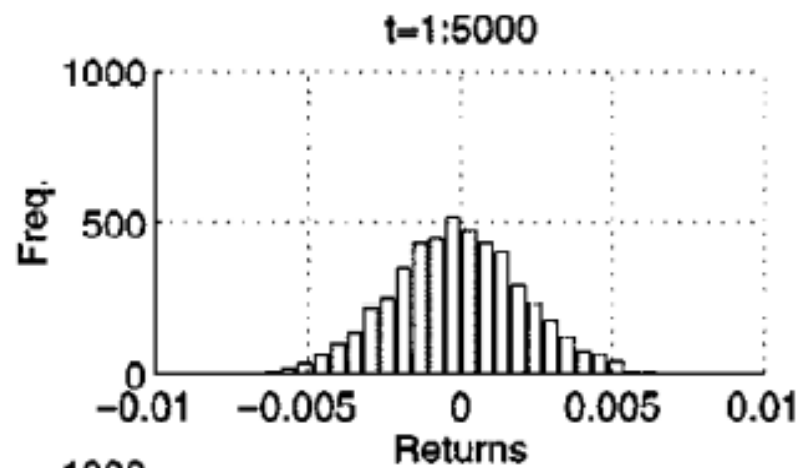
$$F_t = \text{sign}(uF_{t-1} + v_0r_t + v_1r_{t-1} + \cdots v_7r_{t-7} + w)$$

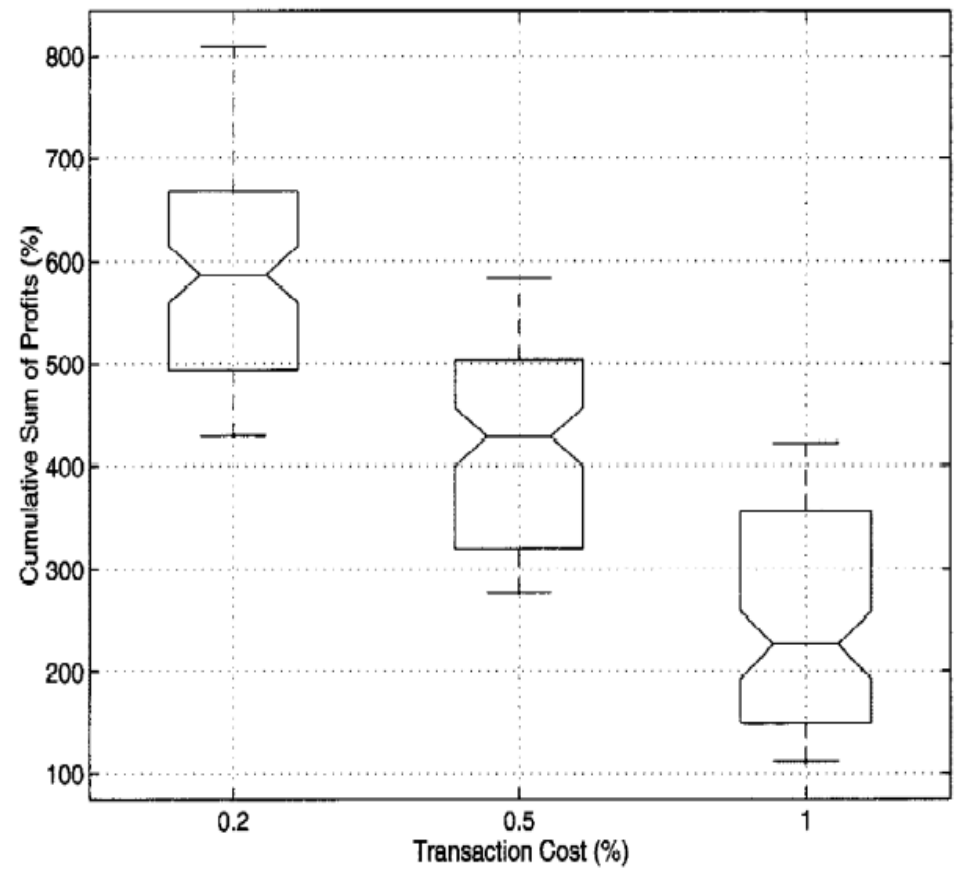
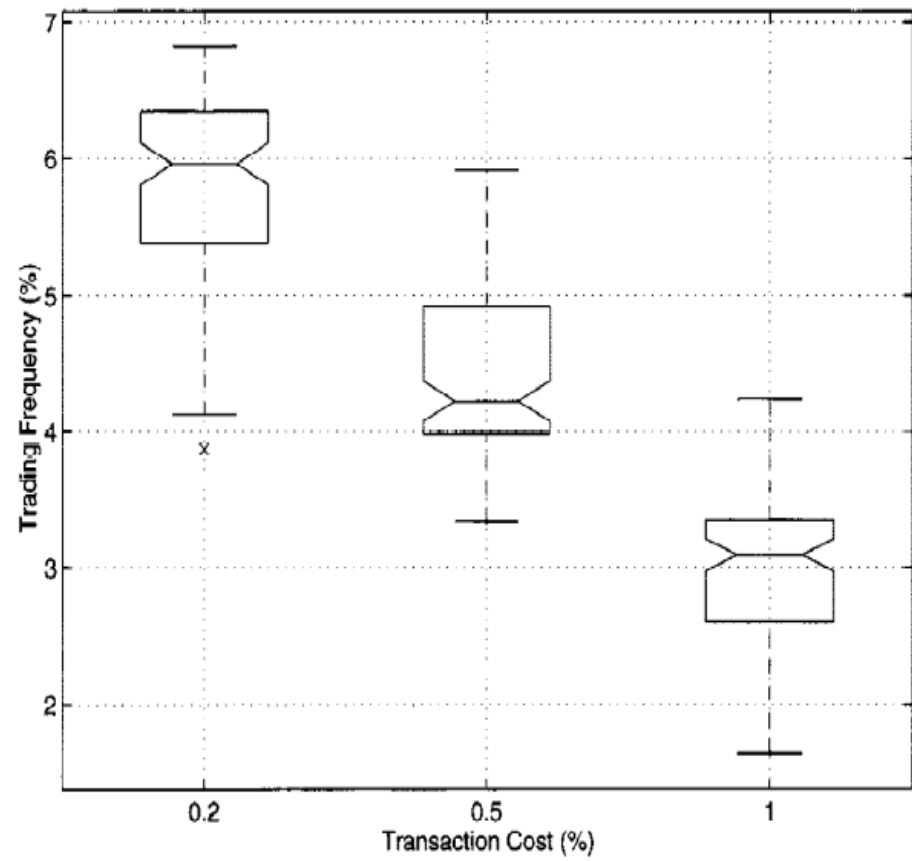
Transaction cost: $\delta = 0.5\%$

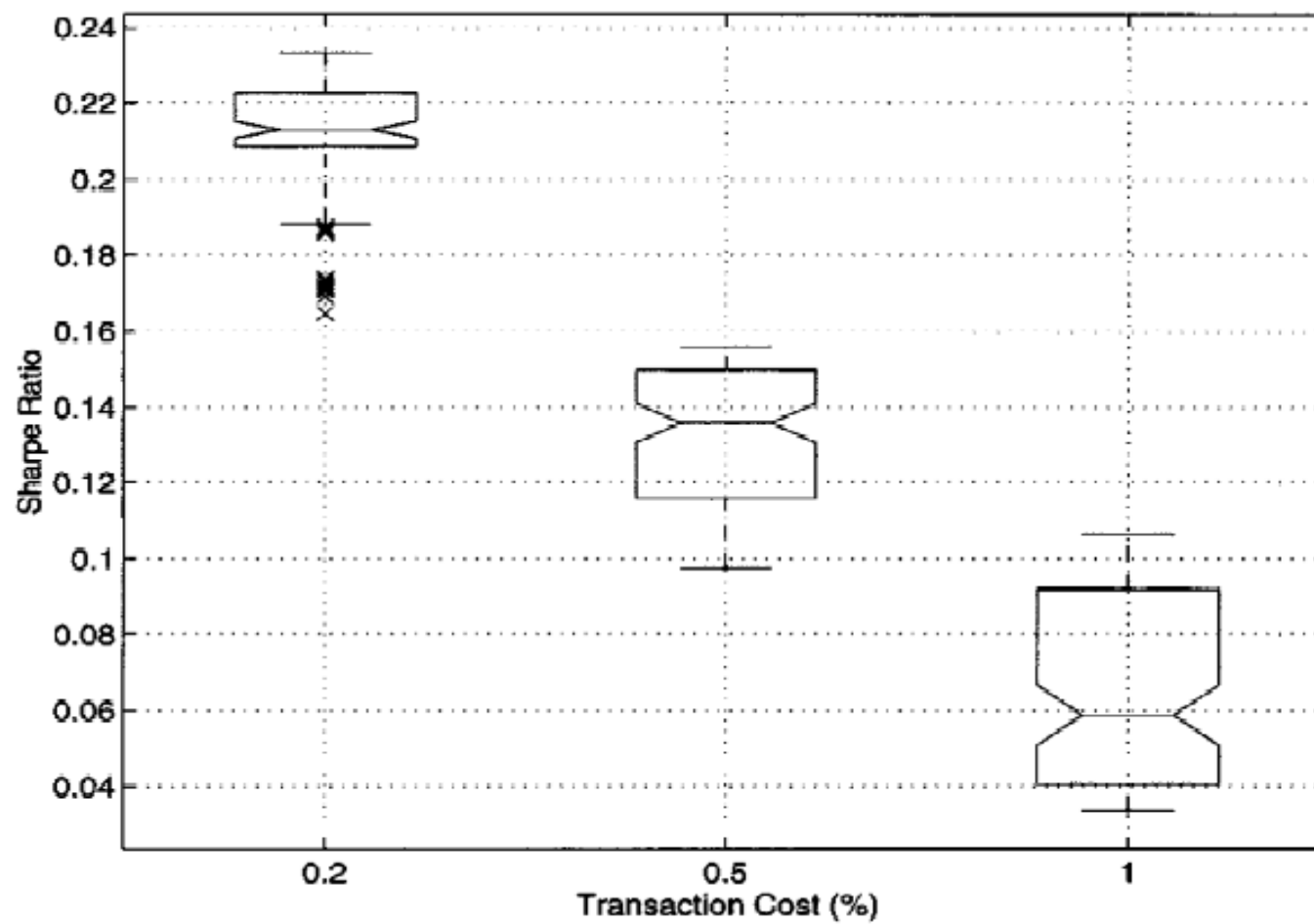
Learns to maximize the Differential Sharpe Ratio

Sharpe ratio :
$$S_T = \frac{\text{Average}(R_t)}{\text{Standard Deviation}(R_t)}$$









Experiment 2: S&P500 and T-Bill

- Goal : show RRL is better than Q-Learning
- Data : 1970~1994 (25 years)
- System is trained on previous 20 years(sliding window)
- The information It also includes macroeconomic data

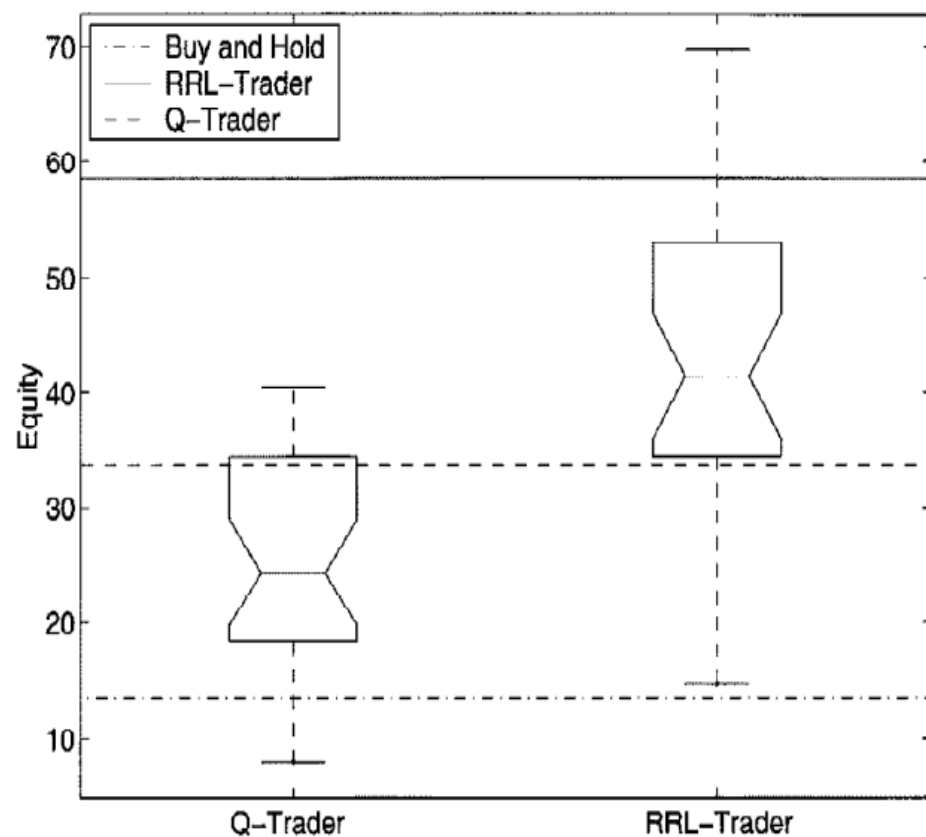
Q-Learning value iteration :

$$Q_{t+1}(x, a) = \max_a \sum_y p_{xy}(a) \{D(x, y, a) + \gamma Q_t(y)\} .$$

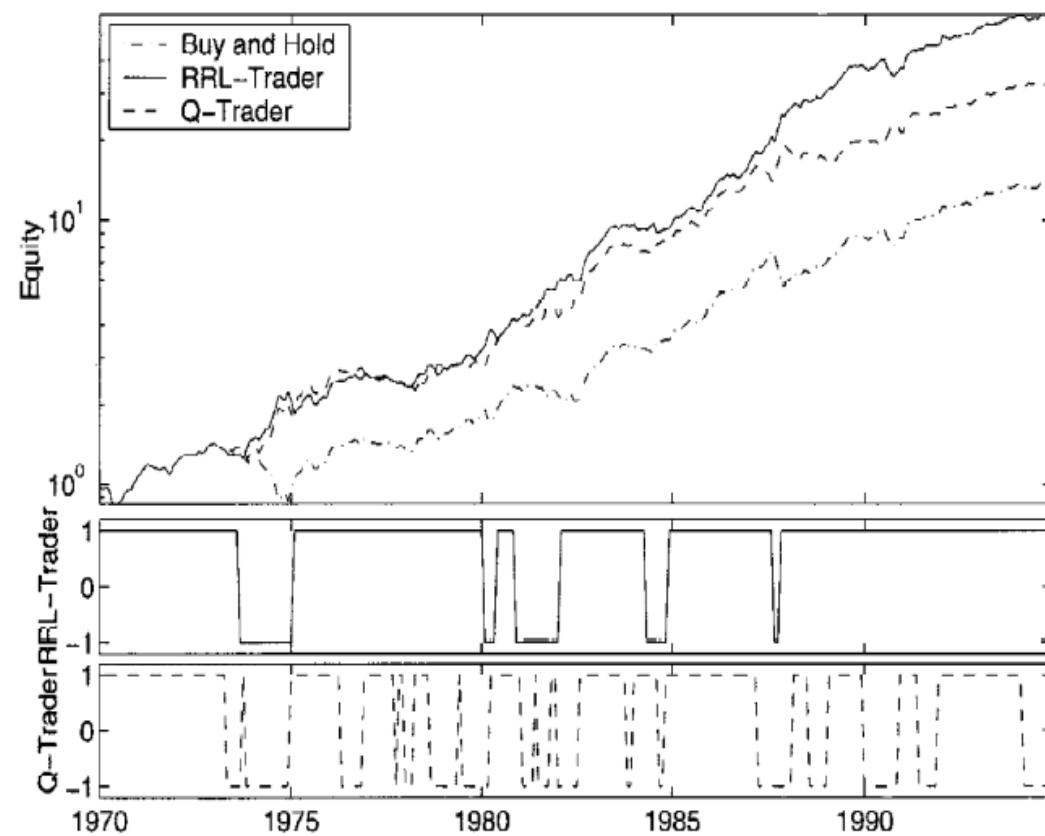
Function approximator :

$$\frac{1}{2} (D(x, y, a) + \gamma \max_b Q(y, b) - Q(x, a))^2 .$$

Final Equity: Q-Trader vs RRL-Trader



RRL-Trader System vs Q-Trader System



Thank you!!!